StaMPS-HPC Manual
V1.0

Mingjia Li

2026.04.02

Department of Earth and Space Science
Southern University of Science and Technology

Contents

Chapter 1

1. Introduction

1.1 What is StaMPS-HPC

StaMPS (Stanford Method for Persistent Scatterers) is a software package that implements an InSAR persistent scatterer (PS) method developed to work even in terrains devoid of man-made structures and/or undergoing non-steady deformation. StaMPS/MTI (Multi-Temporal InSAR) is an extended version of StaMPS that also includes a small baseline method and a combined multi-temporal InSAR method.

StaMPS-HPC (StaMPS for High-Performance Computing) is a performance-optimized derivative of the original StaMPS. It is specifically designed to address the computational bottlenecks, memory overflow issues, high I/O latency, and poor multi-core concurrency often encountered when processing massive, regional-scale InSAR datasets (e.g., over 50 million points). Through a rigorous, ground-up refactoring effort across C/C++, MATLAB, and Bash architectures, StaMPS-HPC transforms the legacy codebase into a modernized, production-ready time-series processor capable of handling large-scale data with unprecedented efficiency.

1.2 Key Features & HPC Optimization

Significant engineering focus was placed on eliminating algorithmic bottlenecks present throughout the entire processing chain. StaMPS-HPC introduces several key innovations:

- **Hybrid Parallel Computing:** The framework integrates **OpenMP** multi-threading across **C/C++** binaries and MATLAB **C-MEX** modules. This allows the software to offload computationally heavy tasks, such as Fast Fourier Transforms (FFTs) and spatial convolutions, directly to compiled C kernels for extreme acceleration.
- **Extreme I/O & Memory Control:** Legacy line-by-line file reading has been

replaced with Block-I/O strategies to minimize disk latency. Furthermore, MATLAB memory thrashing during massive patch merging is solved via a novel "Variable-Centric" parfor architecture.

- **Algorithmic & Architecture Streamlining:** StaMPS-HPC aggressively modernizes the legacy architecture by deprecating outdated pre-processing pipelines (such as ROI_PAC and DORIS), focusing entirely on modern front-ends like ISCE2 and GAMMA. The MATLAB engine has been heavily streamlined, reducing the core scripts from over 120 down to about 60. Slow O(N) loops have been eradicated in favor of native vectorization (`accumarray`, `discretize`), and suffocating complex exponential arithmetic has been refactored into the Real Number Domain.
- **Modernized Ecosystem & UX:** The system enables multi-core concurrent patch extraction seamlessly for ISCE2 and GAMMA front-ends. It also features an overhauled visualization engine that parses over 30 standard GMT Colormaps (.cpt) and introduces new interactive utilities like `ps_export_kmz` and `profile_plot` for high-end data interpretation.

1.3 Acknowledgments & Citations

When using this software for your research or projects, please ensure you properly cite both the HPC software framework and the foundational algorithms.

1.3.1 Acknowledgments

This high-performance derivative framework, StaMPS-HPC, builds upon the foundational Stanford Method for Persistent Scatterers (StaMPS).

- **Original Authors**: We extend our deepest gratitude to Andy Hooper and his colleagues (Stanford University, University of Leeds) for developing the original, pioneering StaMPS software.
- **Scientific Support**: Special thanks to **Jianbao Sun** and **Kejie Chen** for their invaluable support, guidance, and scientific contributions to the underlying concepts of this optimization and refactoring project.

1.3.2 Citing the StaMPS-HPC Framework & Related Research

Since StaMPS-HPC introduces significant architectural refactoring, OpenMP parallelization, and optimization over the original code, please consider citing our published research along with the software repository:

- Li, M., Sun, J., Xue, L., Shen, Z. K., Li, Y., Zhao, B., & Hu, L. (2025). Characterizing aquifer properties and groundwater storage at North China Plain using geodetic and hydrological measurements. Water Resources Research, 61(2), e2024WR037425. https://doi.org/10.1029/2024WR037425
- Li, Mingjia (2026). StaMPS-HPC: High-Performance Parallelized InSAR Time-Series Framework. GitHub repository. https://github.com/limingjia92/StaMPS-HPC

1.3.3 Citing the Original StaMPS Algorithms

Please also reference the foundational papers for the original StaMPS/MTI algorithms developed by Andy Hooper et al.:

- Hooper, A., Bekaert, D., Spaans, K., & Arıkan, M. (2012). Recent advances in SAR interferometry time series analysis for measuring crustal deformation. Tectonophysics, 514, 1-13. doi: 10.1016/j.tecto.2011.10.013.

For details on the inner workings of the algorithms, see Hooper et al.(2004) and Hooper (2007, 2008).

In addition, cite any dependencies used during your pre-processing steps, such as ISCE2 (Rosen et al., 2012), GAMMA (Werner et al., 2000), TRAIN (Bekaert et al., 2015), MCANDIS (Tymofyeyeva and Fialko, 2015), or GACOS (Yu et al., 2018).

1.4 Typographical Conventions

Throughout this manual, you will interact with two different computational environments: the Linux Bash terminal and the MATLAB command window. To help you easily distinguish between the two, we use the following typographical conventions:

- **Linux Bash Commands:** Commands meant to be executed in your standard Linux terminal are presented in standard code blocks without any specific prefix.

```
make_isce_stack_ps.sh 20240905
```

- **MATLAB Commands:** Commands that must be executed inside the MATLAB environment are strictly prefixed with the `>>` symbol.

```
>> plot_patch_kml_isce('INSAR_20240905', 4, 8)
```

- **Variables:** Entries that are specific to your dataset and require your manual modification are enclosed in angle brackets and colored in red (e.g., `<reference_date>`).

Chapter 2

2. System Requirements & Installation

2.1 Hardware Requirements

Given that the extraction of massive point targets, spatial topology construction, and matrix operations consume significant computational resources, StaMPS-HPC places high demands on CPU concurrency, memory capacity, and disk I/O throughput.

- **Minimum Requirements**: An 8-core processor (2.0 GHz or higher), at least 32 GB of RAM, and 1 TB of available disk space.
- **Recommended Requirements**: To achieve optimal processing efficiency and maximize multi-core parallel acceleration, a server or workstation-class processor (32 cores or more, 3.0 GHz or higher) is recommended, along with 128 GB or more of RAM. For storage, a high-speed Solid State Drive (NVMe SSD) of 2 TB or more is preferred, or a large-capacity Hard Disk Drive (HDD) (e.g., 16TB) if you are processing extremely large-scale datasets.

2.2 Core Software Requirements

StaMPS-HPC is developed for Linux environments and utilizes a hybrid architecture of interpreted scripts and compiled binaries. The following core software environments are strictly required:

- **Operating System**: A Linux operating system with a native Bash Shell environment (Ubuntu 18.04/20.04/22.04 LTS or CentOS 7/8 are recommended).
- **MATLAB Engine**: MATLAB (R2020a or later is recommended), including basic data analysis and parallel computing components to support the `parfor` concurrent scheduling and real-domain matrix operations.
- **Compiler Environment**: The GNU Compiler Collection (GCC/G++) must be installed, and it must support the OpenMP multi-threading protocol. This is essential for building the underlying C/C++ accelerated binaries and the MATLAB

C-MEX interface modules. The system also relies on the standard `make` tool for automated source code compilation.

2.3 Supported InSAR Processor Packages

Before executing the time-series processing, StaMPS-HPC requires initial interferometric data stacks generated by upstream InSAR pre-processing software. The following processors are supported:

- **ISCE2**: ISCE is a free and open-source framework designed for the purpose of processing Interferometric Synthetic Aperture Radar (InSAR) data. The source code and installation details can be found at: https://github.com/isce-framework/isce2
- **GAMMA**: The GAMMA Software is a professional, commercial solution for SAR, InSAR, and PSI processing, developed and maintained by GAMMA Remote Sensing. For licensing and installation details, visit: https://www.gamma-rs.ch/gamma-software
- **GMTSAR (Under Development)**: GMTSAR is a free and open-source InSAR processing system designed for users familiar with Generic Mapping Tools (GMT). *Note: The data parsing interfaces and full compatibility for GMTSAR are currently under active development. Complete support will be introduced in upcoming future releases.* The source code and installation details can be found at: https://github.com/gmtsar/gmtsar

2.4 Optional Post-Processing Packages

For atmospheric delay corrections during the post-processing and plotting stages, users may choose to integrate additional external packages or data products. These are optional and not strictly required to run the core StaMPS-HPC pipeline:

- **TRAIN**: The Toolbox for Reducing Atmospheric InSAR Noise (TRAIN) is a free and open-source tool developed in an effort to include current state-of-the-art tropospheric correction methods into the default InSAR processing chain. It can be

downloaded from: https://github.com/dbekaert/TRAIN

- **GACOS**: The Generic Atmospheric Correction Online Service for InSAR (GACOS) is free to use (though not open-source). It utilises the Iterative Tropospheric Decomposition (ITD) model to separate stratified and turbulent signals from tropospheric total delays, and generates high spatial resolution zenith total delay maps. Data products can be downloaded at: http://www.gacos.net/
- **MCANDIS**: MCANDIS is a free and open-source Matlab Code for Atmospheric Noise Depression through Iterative Stacking, a method for mitigating atmospheric noise in InSAR analysis. It can be accessed at: https://zenodo.org/records/4025100

2.5 Download and Compilation

StaMPS-HPC is distributed as source code and relies on a lightweight deployment method. Ensure your machine is connected to the internet and has the required GCC and MATLAB environments configured.

2.5.1 Sourcing the Code

The stable version is hosted on GitHub. Open your Linux terminal and clone the repository to your local machine using git:

```
git clone https://github.com/limingjia92/StaMPS-HPC
```

2.5.2 Automated Compilation

Navigate to the root directory of the software:

```
cd StaMPS-HPC
```

To fully leverage multi-core CPU parallel acceleration, the underlying C/C++ and MATLAB C-MEX modules must be compiled. Execute the following system-level command:

```
make install
```

This command triggers the compilation engine to automatically build the C/C++

source code into standalone binaries with OpenMP support, and drives the MATLAB compiler to build the mixed-acceleration C-MEX dynamic link libraries. If no errors occur and "**StaMPS-HPC installation complete!**" is displayed, the core acceleration engine is successfully deployed.

2.6 Environment Configuration

To seamlessly call the high-performance execution scripts and MATLAB engine from any working directory in your Linux system, you must configure the global environment path variables.

Option 1: Temporary Configuration (Current Terminal Only)

Execute the `source` command in the root directory. This method is only valid for the currently open terminal window:

```
source StaMPS_CONFIG.bash
```

This must be done whenever a new terminal is opened.

Option 2: Permanent Configuration (Recommended)

To make the environment variables globally persistent, modify your user's Bash environment configuration file (typically `~/.bashrc` or `~/.zshrc`). Add the absolute path of the configuration script to the end of the file:

```
source /absolute/path/to/StaMPS-HPC/StaMPS_CONFIG.bash
```

Save the file and run `source ~/.bashrc` to apply the configuration immediately , or simply restart your terminal to let the configuration take effect automatically.

Chapter 3

3. Quick Start Tutorial

3.0 Introduction

To help new users quickly familiarize themselves with the full StaMPS-HPC workflow, this chapter provides a complete, end-to-end quick start tutorial. We will use a lightweight test dataset (located in a subsidence area in eastern Beijing, consisting of 12 scenes and 2 bursts) to demonstrate how to start from raw SLC data, extract Persistent Scatterers (PS) candidates, execute multi-core parallel computing, and finally export the surface deformation time-series results into GIS software.

This tutorial follows the Single-Master PS Mode as the primary demonstration route. All mentioned test datasets, pre-processing results, and intermediate configuration files are available for download from the official Zenodo repository: https://doi.org/10.5281/zenodo.19304714.

3.1 Data Preparation & Pre-processing

Before initiating the core StaMPS-HPC processing, we must prepare the essential interferometric datasets.

3.1.1 Automated Data Download & SLC Preparation

We will use the **autoInSAR** program to fully automate the download of Sentinel-1 data for our test area (Longitude 116.55, Latitude 39.9, covering approximately six months) and generate the corresponding SLC files.

Execute the following command in your Linux terminal:

```
# Automatically download data and generate SLC files using autoInSAR
autoInSAR.py --mode stack --lon 116.55 --lat 39.9 --dlonlat 0.005 --start_date 20180101 --end_date 20180601 --rel_orbit 142 --step all
```

If you wish to clean up redundant data to save disk space, you can execute this optional cleaning command:

Optional: Clean up intermediate data

```
autoInSAR.py --mode stack --lon 116.55 --lat 39.9 --dlonlat 0.005 --start_date 20180101 --end_date 20180601 --rel_orbit 142 --step clean
```

Note: The generated SLC results will be saved in the `process/merged/` directory. Simultaneously, the program will generate spatiotemporal baseline visualization files (`*.png`) and a recommended command list (`stamps_hpc_commands.txt`) in the `results/` directory. For testing convenience, all of these complete results have been packaged and uploaded to Zenodo.

3.1.2 Generating Single-Master Interferograms (PS Mode)

Based on the prepared SLC files, we will use **20180328** as the master image date to generate a series of differential interferograms. Navigate into the process directory and execute:

Navigate to the process directory and generate interferograms

```
make_isce_stack_ps.sh 20180328 merged/SLC merged/geom_reference merged/baselines 20 5
```

Upon completion, the system will generate the `INSAR_20180328` directory. You can check the `INSAR_20180328/Previews_PNG/` folder to view the multi-looked visualization results of the interferometric amplitude and phase. This allows for a quick visual quality control check of the interferometric processing. (**Note:** This folder is also available in the Zenodo repository).

3.2 Spatial Grid Setup & Patch Extraction

When processing massive InSAR datasets, a core advantage of StaMPS-HPC is

its ability to partition data spatially into patches. This significantly reduces memory pressure and enables efficient parallel computing.

3.2.1 Spatial Grid Partitioning & Validation

First, launch MATLAB and use the built-in function to visualize the patch partitioning scheme. In this example, we divide the data into 4 rows and 2 columns, with an overlapping area of 400x200 pixels:

```
>> plot_patch_kml_isce('INSAR_20180328', 4, 2, 400, 200)
```

This command will generate `patches_ISCE.kml` and `patches_ISCE.txt` in your current directory. You can import the KML file into Google Earth or other GIS software to verify the grid layout. (**Note:** These two files are also available in the Zenodo repository within the `results/` folder).

3.2.2 PS Candidates Extraction

Once the grid layout is confirmed, return to your Linux terminal to execute the PS point extraction. The system will extract the binary data based on an amplitude dispersion threshold of 0.4 and write the results into a new `StaMPS_PS` directory:

```
prep_stamps_isce.sh INSAR_20180328 StaMPS_PS 0.4 4 2 1 400 200
```

Upon completion, multiple `PATCH_*` subdirectories will be generated inside the `StaMPS_PS` folder. These directories contain the binary information required for all subsequent time-series processing.

3.3 Core StaMPS-HPC Processing

Now, navigate into the newly generated `StaMPS_PS` directory and launch MATLAB. We will execute the core processing sequentially from Phase 1 to Phase 3.

3.3.1 Phase 1: Local Patch Processing (Steps 1 to 5)

In this phase, noise estimation and phase correction are performed independently for each patch. StaMPS-HPC offers flexible execution options:

Option A: Single-Machine Sequential Execution (Recommended for Beginners)

Run the following command directly within MATLAB:

```
>> stamps(1,5)
```

Option B: HPC Cluster Parallel Distribution (Advanced Recommendation)

If you are operating on a server or HPC cluster, you can split the default `patch.list` and submit the jobs in parallel. First, split the list in your Linux terminal:

```
awk '{print > ("patch_split." NR)}' patch.list
```

Then, execute the different patch lists concurrently in the background via your terminal:

```
matlab -nojvm -nodisplay -batch "stamps(1,5, 'patch_split.1')"
matlab -nojvm -nodisplay -batch "stamps(1,5, 'patch_split.2')"
# ... and so on
```

(**Note:** For MATLAB R2018 and earlier versions, please use the following syntax: `matlab -nojvm -nodisplay -nosplash -nodesktop -r "stamps(1,5, 'patch_split.1');quit;"`)

3.3.2 Phase 2: Patch Merging (Step 5.5)

Once you have ensured that all patches have successfully completed Step 5, execute the high-performance merging step in MATLAB. This seamlessly splices the local patch data into unified global matrices:

```
>> stamps(5.5,5.5)
```

3.3.3 Phase 3: Global Unwrapping & Post-processing (Steps 6 to 8)

Finally, perform global 3D phase unwrapping, estimate the Spatially-Correlated Look Angle (SCLA) error, and apply spatiotemporal filtering to extract the final deformation signal:

```
>> stamps(6,8)
```

3.4 (Optional) Atmospheric Correction

To further mitigate atmospheric delays (especially when analyzing subtle surface subsidence), you can introduce external atmospheric products (such as GACOS). Your test dataset includes a GACOS_data folder.

In MATLAB, you can easily calculate the atmospheric correction using the built-in interface:

```
>> ps_gacos2tca('../GACOS_data')
```

(Alternatively, you can choose to process this using the TRAIN software by executing `aps_weather_model('gacos',1,3)`.)

3.5 Visualization & GIS Export

Upon completing the processing, use the powerful StaMPS-HPC visualization engine to conduct interactive analysis on the deformation results.

3.5.1 Setting the Reference Area & Plotting Velocity

Select a stable area presumed to have zero deformation to serve as your reference (using a stable zone within the test area as an example):

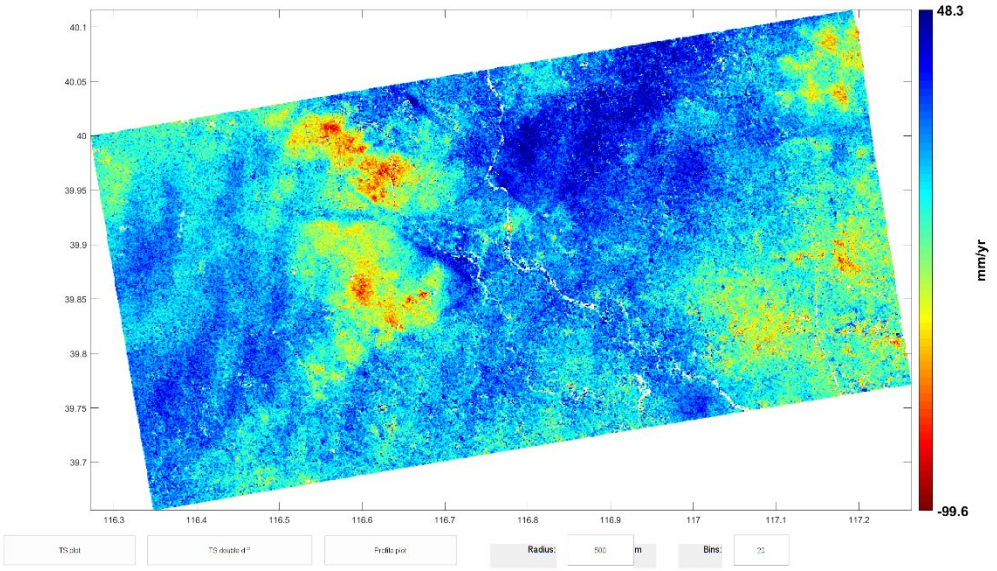
```
>> setparm('ref_lon',[116.38 116.42])
>> setparm('ref_lat',[39.75 39.79])
```

Plot the mean deformation velocity map with DEM and orbital errors removed, and launch the interactive time-series tools (which include TS plot, TS double diff, and Profile plot):

```
>> ps_plot('v-do','ts')
```

If you completed the atmospheric correction in Section 3.4, you can dynamically subtract the atmospheric error (`a_gacos`) while plotting:

```
>> ps_plot('v-dao','a_gacos','ts')
```



Output of the `ps_plot` command. The main window displays the purified deformation velocity field after dynamic error subtraction, while the bottom panel provides one-click access to the single-point, double-difference, and swath profile analysis tools.

3.5.2 Exporting to GIS Platforms (KMZ)

For geospatial overlay analysis, first silently save the currently rendered plot data (the unit, mm/yr, will be saved in the `units` variable):

```
>> ps_plot('v-do',-1)
```

Next, load the spatial coordinates and velocity data into memory, and call the export module to generate a transparent KMZ layer:

```
>> load('ps_plot_v-do.mat','disp_velocity')
>> load('ps2.mat','lonlat')
>> ps_export_kmz('Velocity_Results.kmz', lonlat, disp_velocity, 'opacity', 0.8)
```

The generated `Velocity_Results.kmz` file can be dragged directly into Google Earth for 3D viewing to verify the final surface monitoring results. (**Note:** This file is also available in the Zenodo repository within the `results/` folder).

Chapter 4

4. Pre-processing & PS/SDFP Candidates Extraction

4.1 Pre-processing with ISCE2

This section assumes that you have successfully generated a coregistered SLC stack using the ISCE2 stack processor (e.g., via `stackSentinel.py` command). After this process, the amplitude and phase information (SLC data) for each acquisition is ready.

4.1.1 Important Concept: `data_dir` vs `work_dir`

To ensure strict data management and prevent accidental overwriting, StaMPS-HPC strictly separates the pre-processed interferometric data from the subsequent time-series processing results.

- **Data Directory** (`data_dir`): The directory containing your prepared SLCs and generated interferograms. This directory is created during this pre-processing step.
- **Working Directory** (`work_dir`): The directory where StaMPS-HPC will store the extracted PS (Persistent Scatterer) or SDFP (Slowly-decorrelation Filtered Phase) candidates and all subsequent time-series analysis results.

To avoid the physical copying of massive original datasets common in traditional methods, StaMPS-HPC utilizes system-level symbolic links and VRT/XML header encapsulation to manage the large SLC image archive. Depending on your interferometric network strategy, please choose and run the corresponding encapsulation script below:

4.1.2 For PS (Single Reference) Mode:

For the single reference mode, use the `make_isce_stack_ps.sh` script to link the data stack to a specified single reference image.

`make_isce_stack_ps.sh <reference_date> [slc_stack_path] [stack_geom_path] [slc_stack_baseline_path] [rg_looks] [az_looks]`

Parameters:

- `<reference_date>` (Required): The date of the single reference image (e.g., 20240905).
- `[slc_stack_path]` (Optional): Path to the coregistered SLC directory (Default: `merged/SLC`).
- `[stack_geom_path]` (Optional): Path to the spatial reference geometry directory (Default: `merged/geom_reference`).
- `[slc_stack_baseline_path]` (Optional): Path to the spatial baselines directory (Default: `merged/baselines`).
- `[rg_looks] / [az_looks]` (Optional): Range (Default: 20) and azimuth (Default: 5) looks used to generate Quicklook visualization results.

Output Structure:

Running this script will automatically generate a `data_dir` named `INSAR_<reference_date>` (e.g., `INSAR_20240905`). Inside this directory, individual subfolders for each secondary date (e.g., `20241222`) will be created to store the generated differential interferograms and quicklook files.

4.1.3 For SBAS (Small Baseline) Mode:

For the small baseline mode, use the `make_isce_stack_sbas.sh` script to construct the interferometric pair network.

`make_isce_stack_sbas.sh <neighbor_count> [slc_stack_path] [stack_geom_path] [slc_stack_baseline_path] [rg_looks] [az_looks]`

Parameters:

- `<neighbor_count>` (Required): Number of temporal neighbors to connect (e.g., 2 or 3, determining the density of the interferometric pairs).

- *(The default values for the remaining path and looks parameters are the same as in the PS mode).*

Output Structure:

Running this script will automatically generate a `data_dir` named `SMALL_BASELINES`. Inside this directory, subfolders named after each interferometric pair (e.g., `20240905_20240917`) will be created to store the generated differential interferograms and quicklook files.

4.1.4 Automated Quality Control & Visualization

After the encapsulation and interferogram generation process is complete, it is highly recommended to visually inspect the results. This quick check is crucial for verifying whether the interferometric fringes are forming correctly and for identifying any potential large-scale decorrelation or coregistration errors.

Automated PNG Previews (Recommended)

To make quality control significantly more convenient, both the `make_isce_stack_ps.sh` and `make_isce_stack_sbas.sh` can automatically extract and render ready-to-view PNG images for both the amplitude and wrapped phase of every interferometric pair.

These images are centrally compiled into a dedicated `Previews_PNG` folder within your newly created `data_dir`:

- **For PS Mode**: `INSAR_<reference_date>/Previews_PNG/`
- **For SBAS Mode**: `SMALL_BASELINES/Previews_PNG/`

Inside this folder, you will find perfectly named pairs of files for rapid browsing using any standard image viewer:

- `IFG_AMP_*.png`: The amplitude image preview.
- `IFG_PHA_*.png`: The wrapped interferometric phase preview.

Interactive Inspection with mdx.py (Advanced)

If you wish to interactively probe the phase values or dynamically adjust the color scales of the raw binary data, you can still utilize ISCE2's built-in visualization utility, `mdx.py`, to examine the quicklook.int files located in the individual date subfolders.

- **For PS Mode**:
  - `mdx.py InSAR_path/<secondary_date>/quicklook.int`
- **For SBAS Mode**:
  - `mdx.py SMALL_BASELINES/<date1>_<date2>/quicklook.int`

4.2 Pre-processing the GAMMA

Because GAMMA is a highly flexible professional software, different users often have their own unique processing workflows and command sets when generating interferograms. Therefore, StaMPS-HPC does not intervene in GAMMA's front-end processing.

Preparing your `data_dir`:

Consistent with the design philosophy introduced in Section 4.1, StaMPS-HPC strictly separates pre-processed data from time-series results. For GAMMA datasets, you must manually construct a **Data Directory** (`data_dir`) to store your outputs. You do not need to perform complex data format conversions; our HPC scripts can read these binary data natively as long as they are placed correctly.

Please ensure the contents inside your `data_dir` strictly follow the **Expected Data Directory Structure** below. Note the structural difference between PS and SBAS modes:

```
<data_dir>/                          # Your chosen name for the data directory
    ├── geo/                         # Geometry folder (Shared for PS and SBAS)
    │       ├── *diff_par                    # Diff parameter file
    │       ├── *dem.rdc                     # Radar coordinates DEM
    │       ├── YYYYMMDD.lon                 # Longitude (Radar Coords)
    │       └── YYYYMMDD.lat                 # Latitude (Radar Coords)
    │
    ├── rslc/                        # (For PS Mode Only)
    │       ├── *.rslc                       # Coregistered SLC files
    │       └── *.rslc.par                   # SLC parameter files
```

```
├── diff0/                          # (For PS Mode Only)
│   ├── *.diff                      # Differential interferograms
│   └── *.base                      # Baseline files
│
└── SMALL_BASELINES/                # (Required for SBAS Mode Only)
    ├── rslc/
    │   ├── *.rslc                  # Coregistered SLC files
    │   └── *.rslc.par              # SLC parameter files
    └── diff0/
        ├── *.diff                  # Differential interferograms
        └── *.base                  # Baseline files
```

4.3 Pre-processing the GMTSAR

Note: Native data parsing interfaces for GMTSAR outputs are currently under active development. Full compatibility will be introduced in future StaMPS-HPC releases. For the current version, please rely on ISCE2 or GAMMA for your pre-processing workflows.

4.4 Spatial Grid & Coverage Validation

Before initiating the computationally intensive core processing and physically extracting the candidates into multiple patches (which will be detailed in Section 4.5), it is highly recommended to visually validate your spatial grid configuration.

StaMPS-HPC provides specialized MATLAB utility functions that read the geometry files within your `data_dir` and generate geographic boundary files of your patch network. This allows you to intuitively verify if the configured patches completely cover your region of interest and if the overlap settings are appropriate.

Depending on your pre-processing front-end, open your MATLAB environment and execute the corresponding command:

4.4.1 For ISCE2 Datasets:

```
>>  plot_patch_kml_isce('data_dir',  rg_patches,  az_patches,
rg_overlap, az_overlap)
```

(*Example:* `plot_patch_kml_isce('INSAR_20240905', 4, 8, 400, 400)`)

4.4.2 For GAMMA Datasets:

```
>>  plot_patch_kml_gamma('data_dir',  rg_patches,  az_patches,
rg_overlap, az_overlap)
```

(*Example:* `plot_patch_kml_gamma ('GAMMA_data', 4, 8, 400, 400)`)

Parameters:

- '`data_dir`' (Required): The path to your prepared Data Directory containing the pre-processed data and geometry files (e.g., `'INSAR_20240905'` or `'SMALL_BASELINES'`).
- `rg_patches` (Required): The number of patches you intend to split the data into along the range direction.
- `az_patches` (Required): The number of patches you intend to split the data into along the azimuth direction.
- `rg_overlap / az_overlap` (Optional): The number of overlapping pixels between adjacent patches in the range and azimuth directions, respectively. If omitted, the default value is set to 400 pixels.

Output**:**

Upon successful execution, the script will generate a KML file (e.g., `patches_ISCE.kml` or `patches_GAMMA.kml`) and a multi-segment polygon text file (e.g., `patches_ISCE.txt` for GMT plotting) in your current working directory. You can import the `.kml` file directly into Google Earth or any GIS software to inspect the spatial footprint and overlapping regions of each numbered patch.

4.5 Multi-core Patch Extraction

Once your data is prepared in the `<data_dir>` and your spatial grid is validated, the next step is to extract the initial candidates (PS or SDFP) and split them into

manageable patches. This process reads the amplitude and phase data from the `<data_dir>`, performs initial filtering based on amplitude dispersion, and writes the extracted binary candidate information into your designated `<work_dir>`.

Dividing the region into multiple patches (using `rg_patches` and `az_patches`) significantly reduces the memory burden for subsequent single-patch processing steps and enables high-performance parallel computing.

Depending on your pre-processing software, run the corresponding extraction command in your terminal:

4.5.1 For ISCE2 Datasets:

```
prep_stamps_isce.sh <data_dir> <work_dir> [da_thresh] [rg_patches] [az_patches] [jobmaxnum] [rg_overlap] [az_overlap] [maskfile]
```

(*Example:* `prep_stamps_isce.sh INSAR_20240905 StaMPS_PS 0.4 4 8 5 400 400`)

4.5.2 For GAMMA Datasets:

```
prep_stamps_isce.sh <reference_date> <data_dir> <work_dir> [da_thresh] [rg_patches] [az_patches] [jobmaxnum] [rg_overlap] [az_overlap] [maskfile]
```

(*Example:* `prep_stamps_gamma.sh 20240905 GAMMA_data StaMPS_SBAS 0.6 4 8 5 400 400`)

4.5.3 Core Parameters Explained:

- `<data_dir>` (Required): The path to your prepared input data directory (e.g., `INSAR_20240905` or `SMALL_BASELINES`).
- `<work_dir>` (Required): The path to the output working directory where StaMPS-HPC will store the extracted candidates and perform all subsequent time-series analysis (e.g., `StaMPS_PS` or `StaMPS_SBAS`).
- `<reference_date>` (Required for GAMMA only): The date of the single reference image for PS mode, or the designated "super reference" image date for

SBAS mode (e.g., `20240905`).

- `[da_thresh]`: The amplitude dispersion threshold used to select initial candidates. A lower value means stricter selection and fewer points. (Default: 0.4 for PS mode, 0.6 for SBAS mode).
- `[rg_patches] / [az_patches]`: The number of patches to split the data into along the range and azimuth directions. You can use the exact values validated in Section 4.4. (Default: 1).
- `[jobmaxnum]`: The maximum number of parallel background jobs to run concurrently for patch extraction.
  - **Hardware Warning:** This binary extraction process is extremely I/O intensive. If you are using a high-speed Solid State Drive (NVMe SSD), you can set this equal to your physical CPU core count for maximum acceleration. However, if you are using traditional mechanical hard drives (HDD), increasing this number may actually decrease performance due to severe disk thrashing bottlenecks. For HDDs, it is recommended to keep this value as 1~5.
- `[rg_overlap] / [az_overlap]`: Overlapping pixels between adjacent patches in range and azimuth. (Default: 400).
- `[maskfile]`: (Optional) Path to a specific geographic mask file to limit the extraction area.

4.5.4 Advanced: Selective Patch Processing

If a previous extraction run failed on specific patches, or if you discovered (via the spatial grid validation in Section 4.4) that only a subset of the patches actually covers your specific Region of Interest (ROI), you can utilize the Selective Patch Processing feature to save significant computational time and I/O resources.

Before running the extraction command, simply create a plain text file named `patch.list` inside your designated `<work_dir>`. In this file, list the exact names of the patch folders you wish to process, with one folder name per line. For example:

```
PATCH_2
PATCH_3
PATCH_5
```

When you execute the `prep_stamps_` script, the system will automatically detect this `patch.list` file and exclusively process the specified patches, bypassing all others.

4.5.5 Output Files Description

Upon successful execution, the script will generate multiple `PATCH_X` subdirectories within your <work_dir>. Inside each patch folder, you will find the extracted candidate data stored in both text and binary formats. These files are crucial for the subsequent time-series processing steps:

- `pscands.1.ij`: Candidate locations containing pixel row and column indices (Plain text file).
- `pscands.1.ij.int`: The binary format version of the candidate locations (`pscands.1.ij`).
- `pscands.1.da`: Candidate amplitude dispersion values (Binary file).
- `pscands.1.ll`: Longitude and latitude coordinates (Binary file).
- `pscands.1.hgt`: Topographic height/elevation values (Binary file).
- `pscands.1.ph`: Interferometric phase values (Binary file).

(Note: If you are processing GAMMA datasets, the system will additionally generate `pscands.1.head` for heading angles and `pscands.1.inc` for incidence angles).

Chapter 5

5. Core StaMPS-HPC Processing

5.0 Introduction & Global Control

Welcome to the core MATLAB processing engine of the StaMPS-HPC framework. Through a rigorous refactoring effort, the legacy StaMPS codebase has been aggressively streamlined—reducing the core executable functions from over 120 down to 59 essential modules. Computationally suffocating loops have been eradicated and replaced with heavily vectorized matrix operations and OpenMP-accelerated C-MEX integrations.

Whether you are processing Persistent Scatterers (PS) or Small Baselines (SBAS) candidates, once your data is extracted into the `<work_dir>`, StaMPS-HPC uses a **Unified Processing Engine**. The system will automatically detect your data mode and route it through the optimized pipeline.

The Core Control Function: `stamps.m`

The entire time-series processing chain is driven by a single, powerful master script: `stamps.m`. You can control exactly which steps to execute by defining the start and end steps, and optionally passing a patch list for cluster parallelization.

```
>> stamps(<start_step>, <end_step>, ['patch_list_file'])
```

The standard processing workflow is logically divided into three major phases:

1. **Phase 1: Local Patch Processing (Steps 1 to 5)**

These steps process the initial data, estimate phase noise, select final candidates, weed out noisy pixels and correct local look angle errors. This phase runs independently on individual patches.

```
>> stamps(1, 5)                    % Run Steps 1-5 sequentially
>> stamps(1, 5, 'patch.list')      % HPC Mode: Only process patches
```

defined in the list

2. **Phase 2: High-Performance Merging (Step 5.5)**

A dedicated StaMPS-HPC step. It safely seamlessly merges the massive multi-patch data structures into a global workspace, preventing memory overflow using a "Variable-Centric" parfor architecture.

```
>> stamps(5.5, 5.5)
```
% Run the patch merging step ONLY

3. **Phase 3: Global Post-processing (Steps 6 to 8)**

These steps perform 3D phase unwrapping, spatially correlated look angle (SCLA) error estimation, and spatiotemporal filtering. These must be executed globally on the merged dataset.

```
>> stamps(6, 8)
```
% Run global unwrapping and filtering

Global Parameter Management

StaMPS-HPC relies on a centralized parameter configuration file named `parms.mat` located in your `<work_dir>`. You do not need to manually edit this file; instead, use the following built-in MATLAB functions:

1. **Auto-Initialization (`ps_parms_default`)**

If `parms.mat` does not exist when you start processing, StaMPS-HPC will automatically call the `ps_parms_default` function. This intelligently initializes the optimal default parameters based on your InSAR processor (ISCE2 or GAMMA) and processing mode (PS or SBAS), while deliberately omitting obsolete legacy parameters to keep the workspace clean.

2. **Viewing Parameters (`getparm`)**

To view the current value of all parameters, or a specific parameter, use the `getparm` function:

```
>> getparm
```
% Displays all parameters and their current values

```
>> getparm('<param_name>')
```
% Displays the value of a specific parameter

(*Example*: `>> getparm('unwrap_grid_size')`)

3. **Modifying Parameters (`setparm`)**

You can dynamically modify any parameter to fine-tune your processing using the `setparm` function. StaMPS-HPC supports partial string matching, meaning you only need to type enough characters of the parameter name to make it unique.

```
>> setparm('<param_name>', <value>)
```

(*Example*: `>> setparm('unwrap_grid_size', 100)`)

Advanced `setparm` controls:

- Reset to Default: Set the value to `nan` to revert a parameter to its optimal system default.

  (*Example*: `>> setparm('unwrap_grid_size', nan)`)
- Delete a Parameter: Pass `-1` as the third argument to entirely remove a parameter from the configuration.

  (*Example*: `>> setparm('obsolete_param', [], -1)`)

5.1 Phase 1: Patch Processing – Steps 1 to 5

In this first major phase, the system performs local time-series processing. It loads the initial candidates, estimates phase noise, selects the final robust pixels, and corrects for local spatially-uncorrelated errors.

Execution Logic: It is important to understand that Steps 1 through 5 are executed **independently inside each individual patch**. The system must complete all local patch processing before any global merging or unwrapping can occur.

5.1.1 Execution Modes

StaMPS-HPC offers three distinct execution modes to accommodate different hardware environments, ranging from personal workstations to massive supercomputing clusters.

1. **Default Sequential Mode (Single Machine)**

Navigate to your main working directory (`<work_dir>`) and execute the following command in MATLAB:

```
>> stamps(1, 5)
```

How it works: The system automatically reads the default `patch.list` file generated during the extraction phase (Chapter 3). It will sequentially process every `PATCH_*` folder listed in that file, running Steps 1 through 5 for each patch one by one. This is the standard method for a single machine.

2. Single Patch Mode

If you only need to process, test, or re-run one specific patch, you can navigate your terminal directly into that patch's directory (e.g., `cd PATCH_1`) before launching MATLAB, and execute:

```
>> stamps(1, 5)
```

How it works: When executed inside a specific patch folder, the system detects its current path and exclusively processes that single patch, ignoring the global list file entirely.

3. HPC Cluster Distribution Mode (Highly Recommended)

To maximize parallel computing efficiency on massive datasets, StaMPS-HPC introduces a highly flexible list-distribution mechanism. You can manually divide your `patch.list` into multiple smaller text files (e.g., `split_1.list`, `split_2.list`, etc.), with each file containing a subset of the patch names.

From your `<work_dir>`, you can then call MATLAB to process a specific list:

```
>> stamps(1, 5, '<split_list_file>')
```

(*Example*: `>> stamps(1, 5, 'split_1.list')`)

Cluster Scheduling & Parallelization:

- **Multi-Node Distribution:** If you are operating on an HPC cluster, you can embed the MATLAB command above into your cluster's job submission scripts (using scheduling commands like `sbatch` for SLURM, `bsub` for LSF, or `qsub` for PBS/Torque). By submitting different `split_x.list` files to different physical compute nodes, you achieve massive parallelization, drastically reducing the total

processing time.

- **Single-Machine Concurrency:** If you are using a single, high-performance server, it is technically possible to launch multiple MATLAB background jobs simultaneously using different split lists. However, **this is not recommended**. StaMPS-HPC already utilizes multi-threading (OpenMP and parfor) under the hood. Forcing multiple MATLAB engines to run concurrently on a single machine will cause severe CPU contention and I/O bottlenecks, which may ultimately slow down your processing.

5.1.2 Step-by-Step Principles and Parameters

The first 5 steps of StaMPS-HPC are executed independently within each local patch. You can use the `getparm` command to view current parameters and `setparm` to adjust them. Below are the core functions invoked, their underlying principles, and the key parameters for advanced users:

Step 1: Load Data

- **Core Functions:** `ps_load_initial_gamma`, `ps_load_initial_isce`, `sb_load_initial_gamma`, `sb_load_initial_isce`.
- **Principle:** Reads the binary candidate data extracted during the pre-processing stage (such as phase, longitude/latitude, and elevation) and formats them into efficient MATLAB workspace matrices. The system will automatically call the correct script based on your selected processing mode (PS or SBAS) and front-end software (GAMMA or ISCE2).
- **Core Parameters:** This step primarily handles data formatting and generally requires no manual parameter adjustment.

Step 2: Estimate Phase Noise

- **Core Function:** `ps_est_gamma_quick`
- **Principle:** An iterative calculation step. The system applies a spatial bandpass filter to the interferograms to strip away spatially-correlated phase contributions

(e.g., deformation and atmospheric delays), allowing for an accurate estimation of the phase noise level (Gamma) for each candidate pixel.

- **Core Parameters:**
  - `<gamma_max_iterations>`: The maximum number of iterations allowed for the Gamma noise estimation. (Default: `3`).
  - `<filter_grid_size>`: The size of the filtering grid in meters (Default: `50`). A larger grid yields smoother results but may filter out localized deformation features.
  - `<clap_win>`: The CLAP spatial filtering window size in pixels (Default: `32`).
  - `<clap_low_pass>`: The cutoff wavelength for the CLAP spatial low-pass filter in meters (Default: `800`).
  - `<max_topo_err>`: The estimated maximum topographic error threshold in meters, used to assist in phase separation, pixels bigger than value are dropped (Default: `20`).

Step 3: Select Candidates

- **Core Function:** `ps_select`
- **Principle:** Based on the phase noise statistical characteristics estimated in Step 2, the system filters out excessively noisy points. This isolates the final robust pixels (Persistent Scatterers for PS mode, or Slowly-decorrelating Filtered Phase pixels for SBAS mode).
- Core Parameters:
  - `<use_fast_topofit>`: [StaMPS-HPC Core Acceleration] Determines whether to use the highly optimized C-MEX version for topographic fitting (`topofit`). `'y'` = Mex version, fast but little coherence divergence, `'n'` = Matlab version , slow but exact precision. (Default: `'y'`).
  - `<select_method>`: The pixel selection method. Options are `'DENSITY'` or `'PERCENT'`. (Default: `'DENSITY'`).
  - `<density_rand>`: Max acceptable spatial density (per km^2) of random-phase pixels. (Default: `20` for PS mode, `2` for SBAS mode).

- `<percent_rand>`: The maximum percentage of pixels allowed to be pure random phase. (Default: `20` for PS mode, `1` for SBAS mode).
- `<gamma_stdev_reject>`: Rejects unstable pixels with excessively high Gamma standard deviations. (Default: `0`, meaning no extra rejection is performed).

Step 4: Weed Out Adjacent Pixels

- **Core Function:** `ps_weed`
- **Principle:** Performs a secondary cleaning process to weed out noisy pixels affected by sidelobe interference from adjacent strong scatterers, or pixels with highly unstable phase over time. This ensures the absolute purity of the final extracted point set.
- **Core Parameters:**
  - `<weed_standard_dev>`: The threshold for weeding out pixels based on standard deviation. Pixels exceeding this threshold are discarded. (Default: `1.0` for PS mode, `1.2` for SBAS mode).
  - `<weed_time_win>`: The time window (in days) used when evaluating the smoothness of the time series. (Default: `730`).
  - `<weed_max_noise>`: The absolute maximum phase noise hard threshold allowed. (Default: `inf`).
  - `<weed_zero_elevation>`: Drops pixels with zero elevation, which are usually located over the sea or invalid areas. (Default: `'n'`).
  - `<weed_neighbours>`: Spatially weeds out redundant neighbouring pixels, keeping only the one with the highest coherence. Note: This is computationally intensive. (Default: `'n'`).
  - `<drop_ifg_index>`: A list of interferogram indices to exclude/drop from processing (e.g., `[12, 15]`). This is extremely useful if quality control reveals specific interferograms suffer from severe decorrelation. (Default: `[]`).

Step 5: Phase Correction

- **Core Function:** `ps_correct_phase`
- **Principle:** Estimates and corrects the spatially-uncorrelated look angle error (such as local fluctuations caused by DEM residual errors) within each local patch. This prepares the wrapped phase for seamless global merging and 3D unwrapping.
- **Core Parameters:** This step relies on parameters already defined in previous steps and requires no specific manual adjustments here.

5.2 Phase 2: Patch Merging – Step 5.5

After successfully completing the local processing (Steps 1 to 5) for all your designated patches, the data must be merged into a single, unified global workspace before performing 3D phase unwrapping.

An Independent Merge Mechanism

This dedicated merge step is a unique architectural design exclusive to StaMPS-HPC. In traditional InSAR processing, directly merging massive multi-patch datasets often triggers severe memory overflow (Out-of-Memory) errors. To completely resolve this bottleneck, StaMPS-HPC strictly isolates the merging process into a standalone step.

5.2.1 Execution & Parameters:

Ensure your terminal is in your main working directory (`<work_dir>`), launch MATLAB, and execute the following command:

```
>> stamps(5.5, 5.5)
```

The system will automatically read your default `patch.list` file, extract the names of all the successfully processed `PATCH_*` folders, and begin the merging sequence.

Underlying Optimization (How it Works):

Under the hood, this step sequentially invokes the `ps_merge_patches` and `ps_calc_ifg_std` functions. StaMPS-HPC utilizes a novel "Variable-Centric"

`parfor` architecture. It seamlessly and safely splices the massive data structures from individual patches into a unified global matrix, laying the optimal groundwork for the upcoming global unwrapping phase.

Core Parameters:

- `<merge_resample_size>`: The grid size (in meters) used to resample the pixels during the merge process. Setting this value to `0` means no resampling will occur, preserving the original resolution. (Default: `0` for PS mode, `100` for SBAS mode).
- `<merge_standard_dev>`: The standard deviation threshold used to drop unstable resampled pixels during the merge. Pixels exceeding this threshold will be discarded. (Default: `inf`, meaning no hard threshold is applied by default).

5.2.2 Quality Control: Baseline Visualization

Once Phase 2 has successfully merged the data (generating the ps2.mat file in your `<work_dir>`), it is highly recommended to visually inspect the spatiotemporal baseline network before proceeding to 3D phase unwrapping.

StaMPS-HPC provides a dedicated MATLAB utility specifically for this purpose. Ensure your terminal is in your main working directory (`<work_dir>`), launch MATLAB, and execute the following command:

```
plot_baselines
```

Output:

The script automatically reads the merged topology and generates a 2D baseline network graph.:

- Visual Elements: The X-axis represents the Acquisition Date, and the Y-axis represents the Perpendicular Baseline (in meters). The designated Master image is prominently highlighted with a red star, and all image nodes are labeled with their 8-digit acquisition dates.
- Network Analysis: The connecting lines illustrate your valid interferometric pairs

(adapting automatically to either PS or SBAS topologies). Use this plot to confirm that your network is fully connected without isolated images, and to identify any extremely large perpendicular baselines that might cause decorrelation.

- Auto-Export: Upon execution, a high-resolution image named `baseline.png` is automatically generated and saved in your `<work_dir>` for easy reference.

5.3 Phase 3: Global Unwrapping & Post-processing – Steps 6 to 8

In this final processing phase, StaMPS-HPC shifts from local patch-level calculations to global spatial-temporal analysis. This phase unwraps the phase, estimates remaining topographic and orbital errors, and isolates the final deformation signal from atmospheric noise.

5.3.1 Execution Logic & Hardware Warning:

It is absolutely critical that this phase is only executed after Step 5.5 has successfully merged your data. Because these steps operate simultaneously on the entire, seamless global dataset, they are highly memory-intensive (RAM). Please ensure your workstation or HPC node has sufficient memory allocated before initiating this phase.

Execution Command:

Ensure your terminal is in your main working directory (`<work_dir>`), launch MATLAB, and execute:

```
>> stamps(6, 8)
```

5.3.2 Step-by-Step Principles and Parameters

Step 6: Unwrap Phase

- **Core Function:** `ps_unwrap` (which internally drives the `snaphu` engine).
- **Principle:** Performs 3D phase unwrapping on the global point cloud to resolve the 2π phase ambiguity. It utilizes the statistical cost functions and temporal characteristics derived in the earlier steps.
- **Core Parameters:**
  - `<unwrap_method>`: The core algorithm used for unwrapping.

- `'2D'`: Spatial smoothing only. Fastest, but highly susceptible to phase jumps.
- `'3D_QUICK'`: Time-space smoothing with simple thresholding. Fast, but lower accuracy.
- `'3D'`: (Standard) Temporal smoothing combined with iterative optimization. HPC-Parallelized and highly accurate. (Default for **SBAS** mode).
- `'3D_FULL'`: Temporal modeling via local linear regression. Offers the highest precision for Persistent Scatterers but is incompatible with SBAS. **(**Default for **PS** mode**)**.

- `<unwrap_prefilter_flag>`: Determines whether to prefilter the phase before unwrapping. (Default: `'y'`, highly recommended).
- `<unwrap_grid_size>`: The resampling grid spacing (in meters) if the prefilter is enabled. (Default: `200`).
- `<unwrap_gold_n_win>`: The window size used for the Goldstein spatial filter. (Default: `32`).
- `<unwrap_time_win>`: The temporal smoothing window (in days) used to estimate the phase noise distribution. (Default: `730`).
- `<subtr_tropo>` / `<tropo_method>`: Whether to subtract an external tropospheric estimate before unwrapping (`'y'` or `'n'`, Default: `'n'`), and the method used (e.g., `'a_l'`, `'a_e'`).
- `<unwrap_hold_good_values>`: Hold good pixels from first unwrap, only work for SBAS mode. (Default: `'n'`).

Step 7: Calculate Spatially-Correlated Look Angle (SCLA) Error

- **Core Functions:** `ps_calc_scla`, `ps_smooth_scla`
- **Principle**: Estimates and removes Spatially-Correlated Look Angle (SCLA) errors from the unwrapped phase. These errors primarily consist of residual DEM (topographic) errors and master image orbit inaccuracies.
- Core Parameters:

- `<scla_drop_index>`: A list of interferogram indices (e.g., `[12, 15]`) to explicitly exclude from the SCLA calculation if they are known to contain severe unwrapping errors or extreme turbulence. (Default: `[]`).
- `<scla_deramp>`: Whether to estimate and remove a spatial phase ramp for each interferogram. (Default: `'n'`).
- `<scla_method>`: The mathematical method used for estimating SCLA. (Default: `'L2'`).

Step 8: Filter Spatially-Correlated Noise

- **Core Functions:** `ps_scn_filt`, `ps_scn_filt_krig`
- **Principle:** Applies rigorous spatiotemporal filtering to the corrected phase. This step separates the Atmospheric Phase Screen (APS) and other high-frequency spatial noise from the actual, underlying ground deformation signal.
- **Core Parameters:**
  - <scn_kriging_flag>: Whether to use Kriging-based spatial interpolation for SCN filtering. Note: This provides superior accuracy but is heavily computationally expensive. (Default: `'n'`).
  - `<scn_wavelength>`: The wavelength (in meters) of the spatially correlated noise to be filtered. (Default: `100`).
  - `<scn_time_win>`: The time window (in days) used for Spatial-Correlated Noise (SCN) estimation. (Default: `730`).
  - `<scn_deramp_ifg>`: Indices of specific interferograms where deramping should be forcefully applied during filtering. (Default: `[]`).
  - `<krig_atmo>`: Whether to estimate the Atmospheric Phase Screen specifically using spatial Kriging. Also computationally expensive. (Default: `'y'`).

5.3.3 Advanced Technique: Iterative Unwrapping & Noise Reduction

During the 3D phase unwrapping process in Step 6 (`ps_unwrap`), the algorithm attempts to pre-subtract several "nuisance terms" from the wrapped phase. Removing

these noisy components beforehand dramatically reduces phase jumps and ensures a much cleaner, more reliable unwrapped result. These pre-subtracted terms can include:

- **Recursive Unwrapping:** Controlled by `<unwrap_hold_good_values>`, which identifies and holds highly reliable pixels (such as those passing SBAS phase closure checks).
- **Tropospheric Delay (APS)**: Controlled by `<subtr_tropo>` and `<tropo_method>`, which subtracts the estimated Atmospheric Phase Screen.
- **Spatially Correlated Look Angle (SCLA) Error**: Subtracts topographic residuals and orbital ramps.

The Iterative Workflow:

There is an inherent paradox in the standard workflow: Step 6 requires SCLA to improve unwrapping, but SCLA is only calculated afterward in Step 7. To overcome this and achieve superior accuracy, advanced users are highly recommended to use an Iterative Unwrapping approach:

1. **First Pass:** Execute `>> stamps(6, 7)`. The system will unwrap the raw phase (without SCLA subtraction) and then calculate the initial SCLA in Step 7.
2. **Second Pass:** Execute `>> stamps(6, 7)` again. During this second run, Step 6 will automatically detect the SCLA results generated from the first pass, subtract this noise from the wrapped phase before unwrapping, and yield a significantly improved unwrapped result. Step 7 will then compute a refined, residual SCLA.

This two-pass iterative method generally produces a noticeably smoother and more accurate deformation time-series than a single default execution.

Resetting the Process:

If you ever need to completely restart the unwrapping process from scratch (e.g., after changing core filtering parameters or dropping interferograms), leftover SCLA files from previous runs might interfere with the fresh unwrapping.

To safely wipe the slate clean, simply execute the following built-in utility in your MATLAB command window before running Step 6 again:

```
>> scla_reset
```

This command acts as a one-click cleanup tool, instantly removing all existing SCLA result files from your `<work_dir>` and ensuring a pristine environment for your new unwrapping attempt.

5.4 Advanced Workflow: Combined PS/SBAS Merging (MTI)

After independently completing the patch merging for both your PS and SBAS datasets, StaMPS-HPC offers an advanced Multi-Temporal InSAR (MTI) workflow. This powerful feature allows you to physically merge the Persistent Scatterers (PS) with the Slowly-decorrelating Filtered Phase (SDFP) pixels from your SBAS processing. The result is a unified dataset with an exceptionally high spatial point density, which significantly enhances the reliability of the subsequent 3D phase unwrapping and the overall accuracy of the final deformation field.

5.4.1 Execution Logic & Prerequisites:

This merging operation must be executed only after both your independent PS and SBAS datasets have successfully completed **Phase 2** (i.e., after running `stamps(5.5, 5.5)` in both directories). Once the new merged directory is created, you will then execute **Phase 3** (Global Unwrapping & Post-processing) uniformly within that new directory.

Execution Command:

In your MATLAB environment, call the `ps_sb_merge` function to execute the combination:

```
>> ps_sb_merge('<ps_wd>', '<sb_wd>', '[merged_wd]')
```

Core Parameters Explained:

- `<ps_wd>` (Required): The file path to your PS working directory (where Phase 2 has been completed).

- `<sb_wd>` (Required): The file path to your SBAS working directory (where Phase 2 has been completed).
- `[merged_wd]` (Optional): The file path for the output directory where the merged results will be saved. If left blank, the system will automatically create a folder named `MERGED` in your current working directory.

5.4.2 Standard MTI Workflow Steps:

1. **Independent Processing:** Run the full StaMPS-HPC pipeline independently in two separate working directories (one for PS, one for SBAS) up until the merging step completes: `>> stamps(1, 5.5)`.
2. **Execute Data Merging:** Call the `ps_sb_merge` script. The system will automatically identify, align, and splice the data structures from both directories, generating a new `<merged_wd>` containing the combined point cloud.
3. **Global Unwrapping & Filtering:** Navigate your terminal/MATLAB into the newly generated `<merged_wd>` directory and execute the final Phase 3 processing: `>> stamps(6, 8)`. The system will now perform 3D phase unwrapping on this high-density, unified dataset.

5.4.3 StaMPS-HPC Under the Hood (Optimization Highlights):

- **Extreme Memory Efficiency:** The legacy approach of mapping pixels using memory-exhausting dense 2D grid loops has been completely eradicated. Instead, StaMPS-HPC utilizes low-level `ismember` logic for spatial intersection determination, drastically reducing RAM consumption when merging massive point clouds.
- **Vectorized Compute Acceleration:** The script introduces sparse design matrices (`G` matrix) and deep vectorized operations, achieving lightning-fast phase projection and Signal-to-Noise Ratio (SNR)-weighted merging.

Chapter 6

6. Advanced Plotting & Exporting

6.1 Standard Plotting

Upon completing the full time-series processing and error estimation, visualizing the results is the critical next step for analysis. StaMPS-HPC features a high-performance, universal rendering engine called `ps_plot`. This function is used to plot the spatial distribution of phase, mean deformation velocities, and various estimated error components.

Core Plotting Command:

In your MATLAB workspace (within the `<work_dir>`), invoke the visualization engine using the following syntax:

```
>>  ps_plot('<value_type>',  [plot_flag],  [phase_lims],  [ref_ifg], [ifg_list], ['OptionName'])
```

Command Input Arguments:

- `<value_type>` (Required): A string defining the specific data variable to plot (Detailed extensively in Section 5.1.1).
- `[plot_flag]` (Optional): Controls the background plotting mode.
  - `-1` = Save the extracted matrices to a `.mat` file (useful for exporting).
  - `0` = Plot with a black background and longitude/latitude axes.
  - `1` = Plot with a white background and longitude/latitude axes. (Default)
- `[phase_lims]` (Optional): A 1x2 vector specifying the minimum and maximum limits of the colormap (e.g., `[-10 10]`). The units are adaptive based on the plotted data: radians (**rad**) for phase, millimeters per year (**mm/yr**) for velocity, and meters (**m**) for elevation. If omitted, the limits are calculated automatically.
- `[ref_ifg]` (Optional): The interferogram index to use as a temporal reference (`0` = master, `-1` = incremental step-by-step, or any specific interferogram number/index you wish to reference)..

- `[ifg_list]` (Optional): A specific array of interferogram indices you wish to plot. (Default: `[]`, which represents plotting all available interferograms).
- `['OptionName']` (Optional): Additional string flags. For example, `'a_era5'` specifies an external atmospheric correction model (Covered in Section 5.1.4), and `'ts'` enables the interactive time-series UI extraction (Covered in Section 5.2).

6.1.1 The Core Parameter: `value_type`

The `value_type` is the most important argument. It precisely defines what data type is rendered on the screen. Note that the codes differ slightly depending on whether you are plotting Single-Master (SM/PS) or Small Baseline (SBAS) data.

Basic Data Types:

- `'v' / 'vsb'`: Mean Line-of-Sight (LOS) deformation velocity (SM / SBAS).
- `'vs'`: Velocity standard deviation.
- `'u' / 'usb'`: Unwrapped phase (SM / SBAS).
- `'w' / 'p'`: Raw wrapped phase / Filtered wrapped phase.
- `'hgt'`: Topographic elevation (in meters).

Error & Noise Components:

- `'d' / 'dsb'`: Spatially-correlated DEM (topographic) error.
- `'m'`: Master image atmospheric and orbital error.
- `'o'`: Orbital deramp error.
- `'a' / 'asb'`: Stratified topo-correlated atmosphere (TCA).
- `'s'`: Slave-specific spatial error.
- `'i' / 'isb'`: Ionospheric delay (*Under development*).
- `'t' / 'tsb'`: Solid Earth tide (*Under development*).

Advanced Combinations (Dynamic Subtraction):

You can use a hyphen `-` to dynamically subtract error components from your basic data types on the fly. For example:

- `ps_plot('v-do')`: Plots the mean velocity with DEM error (`d`) and orbital ramp error (`o`) removed.

(Note: Capitalizing the first letter of the type, such as `'V-d'`, will force the system to use the **Single-Master** inversion mode for plotting*).*

6.1.2 Plotting Configuration Parameters

Beyond the direct inputs to `ps_plot`, many global visual effects and computational limits are controlled by system parameters. You can adjust these at any time using the `setparm` function:

Reference Area Definition:

In InSAR time-series analysis, deformation is relative. You must define a stable reference area (assumed to have zero deformation) to calibrate the entire deformation field. You can use either of the following two pairs of parameters:

- **Box Method:** Use `<ref_lon>` and `<ref_lat>` together to define a rectangular bounding box (e.g., `[-118.2, -118.0]` and `[34.0, 34.2]`).
- **Radius Method:** Use `<ref_centre_lonlat>` (e.g., `[-118.1, 34.1]`) paired with `<ref_radius>` (in **meters**) to define a circular reference zone.

Orbital Deramping Parameters:

These parameters are utilized by the underlying `ps_deramp` function specifically when you are estimating or subtracting orbital ramp errors (using the `'o'` value type):

- `<deramp_degree>`: The polynomial degree for the orbital ramp removal (e.g., `1` = linear, `2` = quadratic, and more shown in `ps_deramp`).
- `<deramp_mask_lon>` & `<deramp_mask_lat>`: Longitude and latitude vertices of a polygon. This allows you to explicitly exclude areas of known, genuine deformation (like a subsiding basin) from the ramp estimation, preventing the deformation signal from being mistakenly absorbed as an orbital error.

Scatterer Visual & Spatial Controls:

- `<plot_scatterer_size>`: The actual physical size of each scatterer in map units/meters (Default: `120`).
- `<plot_pixels_scatterer>`: The visual size of each scatterer symbol in screen pixels (Default: `3`).
- `<lonlat_offset>`: A manual `[lon, lat]` offset to perfectly align your PS

point cloud with external optical base maps or DEMs. (*Deprecated*: This is a legacy feature retained for backward compatibility).

- `<plot_lon_rg>` & `<plot_lat_rg>`: Specify the exact longitude and latitude display limits. This is highly useful for zooming in and consistently plotting a specific sub-region of your data without rendering the entire footprint.

6.1.3 Advanced Extension: GMT Colormaps

The color palette used for rendering is controlled by the `<plot_color_scheme>` parameter (Default is `'inflation'`, blue for uplift and red for subsidence).

To meet the rigorous standards of publication-quality mapping, StaMPS-HPC deeply integrates the `cptcmap` module, providing native support for standard Generic Mapping Tools (GMT) `.cpt` colormap files. The system includes a built-in `cptfiles` directory, allowing you to seamlessly apply professional palettes:

`>> setparm('plot_color_scheme', 'GMT_relief')` % Ideal for 'hgt' elevation plots

`>> setparm('plot_color_scheme', 'GMT_seis')` % Classic diverging palette for deformation

By leveraging this extension, you can generate stunning, high-contrast, publication-ready figures directly within the MATLAB environment.

6.1.4 External Data Correction (Atmosphere & Ionosphere)

While StaMPS-HPC is highly capable of estimating and filtering spatiotemporal noise internally, incorporating external meteorological or physical models often yields a more robust correction. When you choose to dynamically subtract atmospheric (`'a' / 'asb'`) or ionospheric (`'i' / 'isb'`) errors in your `value_type` (e.g., plotting `v-a` or `u-ai`), you must explicitly tell `ps_plot` which external model dataset to use.

This is achieved by passing a specific string flag as an `['OptionName']` argument in your `ps_plot` command.

1. Tropospheric/Atmospheric Phase Screen (APS) Correction

To apply an external atmospheric correction, append the `'a_<model_name>'` flag

to your command.

% Example: Plot mean velocity with ERA5 atmospheric correction applied

```
>> ps_plot('v-a', 'a_era5')
```

Supported external APS models include:

- `'a_era5'`: Atmospheric correction calculated using the ERA5 global climate model (typically processed and formatted via TRAIN).
- `'a_gacos'`: Zenith total delay correction derived from the GACOS high-resolution products.
- `'a_mcandis'`: Atmospheric correction processed via the customized MCANDIS tool.

Extensibility Note for Advanced Users: If you have processed atmospheric data using other models, you can easily integrate them by modifying the underlying `ps_plot_tca.m` script. For a comprehensive understanding of these external atmospheric correction methods, we highly recommend consulting the official manual for TRAIN (The Toolbox for Reducing Atmospheric InSAR Noise).

2. Ionospheric Phase Screen (IPS) Correction (Under development)

Similarly, for L-band or P-band datasets where ionospheric delay becomes a dominant error source, StaMPS-HPC is building support for external Ionospheric Phase Screen (IPS) corrections.

To apply this (once fully implemented), you would append the `'i_<model_name>'` flag:

% Example: Plot mean velocity with Split-Spectrum ionospheric correction applied

```
>> ps_plot('v-i', 'i_split')
```

Planned supported models:

- `'i_azshift' / 'i_az'`: Ionospheric delay estimated using the Azimuth Shift

method.

- `'i_split' / 'i_s'`: Ionospheric delay estimated using the Split-Spectrum method.
- `'i_tec' / 'i_t'`: Ionospheric delay estimated using external Total Electron Content (TEC) maps.

6.2 Interactive Time-Series Analysis

When rendering your results using `ps_plot`, simply appending the `'ts'` string flag to your command (e.g., `>> ps_plot('v-dao', 'ts', 'a_era5')`) will automatically generate a suite of interactive time-series analysis tools beneath the main figure window.

Behind the scenes, StaMPS-HPC has completely rebuilt the interactive engines (`ts_plot` and `ts_plotdiff`) to guarantee memory safety and robust crosshair tracking across multiple tools. The system also automatically injects the "zero deformation" master reference into your time-series extraction, ensuring physical continuity and strict mathematical rigor for linear velocity fitting.

6.2.1 Single-Point Time-Series (`ts_plot`)

This function extracts and visualizes the absolute historical deformation trend for a single specific location.

How to Use:

1. Click the `TS plot` button located at the bottom of the figure window.
2. Move your cursor over the main plot and click on any pixel of interest.
3. A new figure window will instantly pop up, displaying the complete historical deformation time-series curve for that selected point.
4. **Interactive Feedback:** A red crosshair target will appear on the main plot to precisely anchor your selection, and the exact latitude and longitude coordinates of the point will be printed in your MATLAB command window.

Advanced Tip: Spatial Averaging

If your extracted single-point time-series contains high-frequency noise, you can utilize the `Radius` text box at the bottom of the interface. By entering a search radius in meters (e.g., `500`), the system will automatically average the deformation of all pixels within that radius, effectively smoothing the resulting time-series curve.

6.2.2 Double-Difference Time-Series (`ts_plotdiff`)

This function extracts the relative deformation time-series between two specified locations. It is exceptionally useful for analyzing differential settlement, such as comparing a subsiding landslide body to stable bedrock, or measuring displacement across a geological fault.

How to Use:

1. Click the `TS double diff` button.
2. First, click on a stable **Reference point** within the main plot.
3. Next, click on a deforming **Target point**.
4. A new figure will appear displaying the relative deformation time-series between the two points, accompanied by a small context map illustrating their spatial relationship.
5. **Interactive Feedback:** The main plot will explicitly mark the two endpoints with red and blue crosshairs, respectively, and output their coordinate information to the console. The `Radius` text box applies to this function as well, allowing you to control the spatial averaging extent for both the reference and target points simultaneously.

6.3 Swath Profile Plotting

Just like the single-point and double-difference time-series tools, the interactive profile extraction tool is automatically activated when you append the `'ts'` flag to your `ps_plot` command. This tool allows you to extract a dynamic cross-sectional swath

profile across your deformation field.

To provide maximum analytical value, StaMPS-HPC introduces a modernized **Dual-Y-Axis** design. When you extract a profile, the resulting plot simultaneously maps the topographic elevation curve alongside the deformation scatter points (such as mean velocity). This allows you to instantly visually correlate ground deformation with terrain features (e.g., mountain slopes, valleys, or fault scarps).

How to Extract a Swath Profile

1. Define the Profile Line:

- Click the **Profile plot** button located at the bottom of the main figure window, right next to the time-series buttons.
- On the main plot, click once to define the **Start point** of your profile.
- Click a second time to define the **End point**.

2. Interactive Feedback:

- Once both points are selected, the system will immediately pop up the dual-axis profile figure.
- On the main map, your start and end points will be clearly marked with red and blue crosshairs, respectively, and a solid black line will be drawn connecting them to visualize your exact swath path.

Profile Configuration Parameters

You can precisely control the dimensions and resolution of your swath profile using the text boxes located at the bottom of the interactive interface:

- `<Radius>`**:** This parameter defines the buffer width (in meters) of your swath profile. The system will collect and project all PS/SDFP points that fall within this perpendicular distance on either side of your drawn profile line.
- `<Bins>`**:** This parameter specifies the total number of sampling intervals (bins) along the length of the profile line. Adjusting this value allows you to control the spatial resolution and the visual smoothing of the scattered deformation data within

the plot.

6.4 Exporting to GIS via KMZ

To meet the demands for cross-platform presentation and geospatial analysis, StaMPS-HPC allows you to export your processed data into the KMZ format for intuitive 3D visualization in GIS software such as Google Earth or ArcGIS.

StaMPS-HPC introduces a highly customized rasterization export pipeline (driven by the `ps_export_kmz` core function). This module dynamically interpolates the massive point cloud into a high-resolution PNG overlay with an alpha (transparency) channel, strictly packaging it into a standard OpenGIS-compatible KMZ archive.

Step 1: Export Data Entities from `ps_plot`

Before generating a KMZ, you must extract the physical data from the rendering engine. By appending the `-1` flag to the end of your `ps_plot` command, the system will execute in silent mode (without plotting on the screen) and directly save the currently rendered graphical data to a local `.mat` workspace file.

```
>> ps_plot('v-dao', 'a_era5', -1)
```

Upon execution, the system will generate a data entity file (e.g., `ps_plot_v-dao.mat`) in your working directory.

Step 2: Generate the KMZ File

Next, load the spatial coordinates (`lonlat`) and the exported deformation data (e.g., the deformation velocity `ph_disp`) into your workspace memory, and call the `ps_export_kmz` export module:

```
>> load('ps_plot_v-dao.mat', 'ph_disp')
>> load('ps2.mat', 'lonlat')
>>   ps_export_kmz('Velocity_Results.kmz',   lonlat,   ph_disp,
'opacity', 0.8)
```

Once completed, you will find a file named `Velocity_Results.kmz` in your current directory. You can directly drag and drop this file into Google Earth for seamless 3D viewing.

Core Export Parameters (`ps_export_kmz`)

Command Syntax:

```
>> ps_export_kmz('<filename>', <lonlat>, <data>, ['PropertyName', PropertyValue])
```

- `<filename>` (Required): The name of the output KMZ file (e.g., `'Velocity_Results.kmz'`).
- `<lonlat>` (Required): The matrix containing longitude and latitude coordinates, loaded directly from `ps2.mat`.
- `<data>` (Required): The data vector you wish to render (e.g., `ph_disp`), loaded from the `ps_plot` output file.
- `'opacity'` (Optional): The opacity of the raster overlay, ranging from `0` (completely transparent) to `1` (completely opaque). (Default: `0.8`).
- `'clims'` (Optional): A 1x2 vector controlling the colormap limits (e.g., `[-10, 10]`). If not specified, the system will automatically scale to the minimum and maximum values of your data.
- `'colormap'` (Optional): The colormap name or an RGB matrix used for rendering. (Default: `flipud(jet(64))`, blue for uplift and red for subsidence).
- `'resolution'` (Optional): The spatial resolution of the rasterized grid in degrees. (Default: `0.005`).
- `'max_gap'` (Optional): The maximum gap size (in pixels) allowed to be filled during morphological interpolation. This is extremely useful for automatically closing tiny gaps between scattered points to create a continuous deformation map. (Default: `1`).

Chapter 7

7. Core Output Data Structures

7.0 Introduction

After StaMPS-HPC completes the fully automated processing pipeline and multi-core parallel solving, the deformation information, spatial topology, and error components of the massive point cloud are permanently stored in a standardized binary structure.

To completely resolve the issue of scattered text outputs generated by legacy software when handling massive datasets, StaMPS-HPC serializes the processing results and various separated error components of millions of Persistent Scatterers (PS) into highly efficient, native MATLAB workspace files (`.mat`). This design not only drastically increases read/write throughput but also provides exceptional data compatibility for third-party GIS platforms and professional deformation modeling.

7.1 The Data Dictionary

During the full-chain processing (Steps 1 to 8) and post-processing, the system generates a series of standardized `.mat` data files in your main working directory. Below is a data dictionary detailing the core output files, their corresponding rendering parameters in `ps_plot`, and their physical contents:

File Name	ps_plot Parameter	Contents & Purpose
`ps2.mat`	`hgt`	Contains the global spatial and temporal baselines, including topographic elevation (`hgt`), master/slave image dates, longitude/latitude, and radar incidence angles.
`pm2.mat`, `ph2.mat`, `rc2.mat`	`w, p`	Stores the spatial topology, timelines, and wrapped phases for all PS candidates, including the original wrapped phase (`w`) and the filtered wrapped phase (`p`).
`phuw2.mat`	`u / usb`, `v / vsb`, `vs`	The 3D unwrapped phase and deformation velocity sets generated in Step 6. Contains the absolute unwrapped phase (`u/usb`), the mean Line-of-Sight (LOS) deformation velocity

		(v/vsb), and the fitted velocity standard deviation (vs).
scla_smooth2.mat	d / dsb, m, o	The Look Angle and orbital error components estimated and smoothed in Step 7. Contains the spatially-correlated DEM residual error (d/dsb), the master image-specific atmospheric/orbital error (m), and the orbital deramp phase error (o).
scn2.mat	s	The Spatially-Correlated Noise (SCN) set separated via filtering in Step 8. Contains the slave image-specific spatially-correlated noise (s), primarily used to strip high-frequency interference signals like atmospheric delay from the time-series phase.
tca2.mat, iono2.mat, tide2.mat	a/asb, i/isb, t/tsb	Advanced external delay correction sets calculated by extension modules. Contains the strongly topo-correlated atmospheric delay (a/asb), ionospheric delay (i/isb), and solid Earth tide components (t/tsb).
bp2.mat	N/A	The spatial baseline set. Contains the perpendicular baseline matrix (bperp_mat), used to assist in calculating and separating topographic residual phases.
ps_plot_*-*.mat (e.g., v-dao)	Combinations (e.g., v-dao)	The final, purified deformation datasets exported interactively by the user via ps_plot. Contains the final deformation velocities or millimeter-level time-series sequences after subtracting user-specified error combinations, ready for direct GIS integration.

7.2 Understanding Matrix Dimensions

To ensure memory safety and high-efficiency vectorized computing, the massive time-series InSAR results are rigorously organized into high-dimensional matrix structures. When loading these .mat files for secondary development or custom analysis, please adhere to the following core data dimension conventions:

- **Spatial Vector Dimension (N x 1 or N x 2)**: N represents the total number of final selected PS pixels.
  - For example, the spatial geographic coordinates lonlat and topographic elevation hgt are both N x 2 double-precision matrices.
  - The mean deformation velocity ph_disp is an N x 1 single-precision floating-point vector, which significantly saves memory overhead.
- **Spatiotemporal Sequence Dimension (N x M)**: M represents the number of interferograms or SAR image epochs involved in the time-series inversion.

- For instance, the core phase time-series matrix `ph_uw` is an N x M single-precision matrix. This matrix strictly records the cumulative unwrapped phase for each PS pixel at every radar image acquisition time, serving as the foundation for extracting deformation time-series curves (`ts_plot`).

Chapter 8

8. Change History

8.1 Version 1.0 (Release Date: March 2026)

Version 1.0 marks the inaugural release of StaMPS-HPC. As a comprehensive refactoring of the original StaMPS architecture, this release is designed to eradicate memory bottlenecks when processing massive InSAR datasets (e.g., >50 million points), maximize computational throughput, and fully modernize the user interaction and data export pipelines.

Core Architecture & Parallel Computing

- **Hybrid Parallel Computing**: Deeply integrated OpenMP multi-threading across C/C++ binaries and MATLAB C-MEX modules, offloading computationally heavy tasks (e.g., FFTs, spatial convolutions) directly to compiled C kernels.
- **Extreme I/O Control**: Completely replaced legacy line-by-line file reading with Block-I/O strategies, drastically minimizing disk latency and reducing the execution time of C/C++ core binaries (such as `calamp` and `selpsc_patch`) by 40% to 70%.

Algorithmic Refactoring & Acceleration

- **Streamlined Core Engine**: Refactored and streamlined the legacy MATLAB engine from over 120 scripts down to 59 highly optimized core scripts.
- **Native Vectorization**: Eradicated slow O(N) loops in favor of native vectorized instructions (e.g., `accumarray`, `discretize`) and refactored suffocating complex exponential arithmetic into the Real Number Domain.
- **Benchmark Leaps**: Achieved massive performance gains across the pipeline, including a 22.1x speedup in Step 3 (`ps_select`) and a 7.7x speedup in Step 8 (`ps_scn_filt_krig`).

Workflow Integration & Memory Control

- **Concurrent Data Extraction**: Enabled multi-core concurrent patch extraction (`jobmaxnum`) for ISCE2 and GAMMA pre-processing, significantly reducing data preparation time.
- **Memory-Safe Patch Merging**: Designed a novel "Variable-Centric" `parfor` parallel architecture for massive patch merging (Step 5.5) to fundamentally solve MATLAB memory thrashing issues.

Visualization Engine & GIS Compatibility

- **Modernized Universal Rendering Engine**: Deeply integrated native parsing support for over 30 publication-quality standard GMT Colormaps (`.cpt`), greatly enhancing visual aesthetics and professionalism.
- **Interactive Advanced Utilities**: Introduced modern interactive tools, including a dual-axis topographic-deformation swath profile extraction (`profile_plot`) and double-difference time-series analysis.
- **3D Rasterized Export**: Developed the `ps_export_kmz` module to solve the GIS platform crashes caused by legacy vector KMLs, enabling lightning-fast conversion and export of massive point clouds into high-definition, alpha-transparent KMZ files.